

形参匹配与枚举类型

835

要想初始化一个 enum 对象，必须使用该 enum 类型的另一个对象或者它的一个枚举成员（参见 19.3 节，第 737 页）。因此，即使某个整型值恰好与枚举成员的值相等，它也不能作为函数的 enum 实参使用：

```
// 不限定作用域的枚举类型，潜在类型因机器而异
enum Tokens {INLINE = 128, VIRTUAL = 129};
void ff(Tokens);
void ff(int);
int main() {
    Tokens curTok = INLINE;
    ff(128);                      // 精确匹配 ff(int)
    ff(INLINE);                   // 精确匹配 ff(Tokens)
    ff(curTok);                   // 精确匹配 ff(Tokens)
    return 0;
}
```

尽管我们不能直接将整型值传给 enum 形参，但是可以将一个不限定作用域的枚举类型的对象或枚举成员传给整型形参。此时，enum 的值提升成 int 或更大的整型，实际提升的结果由枚举类型的潜在类型决定：

```
void newf(unsigned char);
void newf(int);
unsigned char uc = VIRTUAL;
newf(VIRTUAL);                  // 调用 newf(int)
newf(uc);                       // 调用 newf(unsigned char)
```

枚举类型 Tokens 只有两个枚举成员，其中较大的那个值是 129。该枚举类型可以用 unsigned char 来表示，因此很多编译器使用 unsigned char 作为 Tokens 的潜在类型。不管 Tokens 的潜在类型到底是什么，它的对象和枚举成员都提升成 int。尤其是，枚举成员永远不会提升成 unsigned char，即使枚举值可以用 unsigned char 存储也是如此。

19.4 类成员指针

成员指针（pointer to member）是指可以指向类的非静态成员的指针。一般情况下，指针指向一个对象，但是成员指针指示的是类的成员，而非类的对象。类的静态成员不属于任何对象，因此无须特殊的指向静态成员的指针，指向静态成员的指针与普通指针没有什么区别。

成员指针的类型囊括了类的类型以及成员的类型。当初初始化一个这样的指针时，我们令其指向类的某个成员，但是不指定该成员所属的对象；直到使用成员指针时，才提供成员所属的对象。

为了解释成员指针的原理，不妨使用 7.3.1 节（第 243 页）的 Screen 类：

836

```
class Screen {
public:
    typedef std::string::size_type pos;
    char get_cursor() const { return contents[cursor]; }
    char get() const;
```

```

        char get(pos ht, pos wd) const;
private:
        std::string contents;
        pos cursor;
        pos height, width;
};

```

19.4.1 数据成员指针

和其他指针一样，在声明成员指针时我们也使用*来表示当前声明的名字是一个指针。与普通指针不同的是，成员指针还必须包含成员所属的类。因此，我们必须在*之前添加 `classname::` 以表示当前定义的指针可以指向 `classname` 的成员。例如：

```

// pdata 可以指向一个常量（非常量）Screen 对象的 string 成员
const string Screen::pdata;

```

上述语句将 `pdata` 声明成“一个指向 `Screen` 类的 `const string` 成员的指针”。常量对象的数据成员本身也是常量，因此将我们的指针声明成指向 `const string` 成员的指针意味着 `pdata` 可以指向任何 `Screen` 对象的一个成员，而不管该 `Screen` 对象是否是常量。作为交换条件，我们只能使用 `pdata` 读取它所指的成员，而不能向它写入内容。

当我们初始化一个成员指针（或者向它赋值）时，需指定它所指的成员。例如，我们可以令 `pdata` 指向某个非特定 `Screen` 对象的 `contents` 成员：

```

pdata = &Screen::contents;

```

其中，我们将取地址运算符作用于 `Screen` 类的成员而非内存中的一个该类对象。

当然，在 C++11 新标准中声明成员指针最简单的方法是使用 `auto` 或 `decltype`：

```

auto pdata = &Screen::contents;

```

使用数据成员指针

读者必须清楚的一点是，当我们初始化一个成员指针或为成员指针赋值时，该指针并没有指向任何数据。成员指针指定了成员而非该成员所属的对象，只有当解引用成员指针时我们才提供对象的信息。

837

与成员访问运算符 `.` 和 `->` 类似，也有两种成员指针访问运算符：`.*` 和 `->*`，这两个运算符使得我们可以解引用指针并获得该对象的成员：

```

Screen myScreen, *pScreen = &myScreen;
// .*解引用 pdata 以获得 myScreen 对象的 contents 成员
auto s = myScreen.*pdata;
// ->*解引用 pdata 以获得 pScreen 所指对象的 contents 成员
s = pScreen->*pdata;

```

从概念上来说，这些运算符执行两步操作：它们首先解引用成员指针以得到所需的成员；然后像成员访问运算符一样，通过对象（`.*`）或指针（`->*`）获取成员。

返回数据成员指针的函数

常规的访问控制规则对成员指针同样有效。例如，`Screen` 的 `contents` 成员是私有的，因此之前对于 `pdata` 的使用必须位于 `Screen` 类的成员或友元内部，否则程序将发生错误。

因为数据成员一般情况下是私有的，所以我们通常不能直接获得数据成员的指针。如果一个像 `Screen` 这样的类希望我们可以访问它的 `contents` 成员，最好定义一个函数，令其返回值是指向该成员的指针：

```
class Screen {
public:
    // data 是一个静态成员，返回一个成员指针
    static const std::string Screen::*data()
    { return &Screen::contents; }
    // 其他成员与之前的版本一致
};
```

我们为 `Screen` 类添加了一个静态成员，令其返回指向 `contents` 成员的指针。显然该函数的返回类型与最初的 `pdata` 指针类型一致。从右向左阅读函数的返回类型，可知 `data` 返回的是一个指向 `Screen` 类的 `const string` 成员的指针。函数体对 `contents` 成员使用了取地址运算符，因此函数将返回指向 `Screen` 类 `contents` 成员的指针。

当我们调用 `data` 函数时，将得到一个成员指针：

```
// data() 返回一个指向 Screen 类的 contents 成员的指针
const string Screen::*pdata = Screen::data();
```

一如往常，`pdata` 指向 `Screen` 类的成员而非实际数据。要想使用 `pdata`，必须把它绑定到 `Screen` 类型的对象上：

```
// 获得 myScreen 对象的 contents 成员
auto s = myScreen.*pdata;
```

19.4.1 节练习

838

练习 19.11: 普通的数据指针与指向数据成员的指针有何区别？

练习 19.12: 定义一个成员指针，令其可以指向 `Screen` 类的 `cursor` 成员。通过该指针获得 `Screen::cursor` 的值。

练习 19.13: 定义一个类型，使其可以表示指向 `Sales_data` 类的 `bookNo` 成员的指针。

19.4.2 成员函数指针

我们也可以定义指向类的成员函数的指针。与指向数据成员的指针类似，对于我们来说要想创建一个指向成员函数的指针，最简单的方法是使用 `auto` 来推断类型：

```
// pmf 是一个指针，它可以指向 Screen 的某个常量成员函数
// 前提是该函数不接受任何实参，并且返回一个 char
auto pmf = &Screen::get_cursor;
```

和指向数据成员的指针一样，我们使用 `classname::*` 的形式声明一个指向成员函数的指针。类似于任何其他函数指针（参见 6.7 节，第 221 页），指向成员函数的指针也需要指定目标函数的返回类型和形参列表。如果成员函数是 `const` 成员（参见 7.1.2 节，第 231 页）或者引用成员（参见 13.6.3 节，第 483 页），则我们必须将 `const` 限定符或引用限定符包含进来。

和普通的函数指针类似，如果成员存在重载的问题，则我们必须显式地声明函数类型以明确指出我们想要使用的是哪个函数（参见 6.7 节，第 211 页）。例如，我们可以声明一

个指针，令其指向含有两个形参的 get：

```
char (Screen::*pmf2)(Screen::pos, Screen::pos) const;
pmf2 = &Screen::get;
```

出于优先级的考虑，上述声明中 Screen::*两端的括号必不可少。如果没有这对括号的话，编译器将认为该声明是一个（无效的）函数声明：

```
// 错误：非成员函数 p 不能使用 const 限定符
char Screen::*p(Screen::pos, Screen::pos) const;
```

这个声明试图定义一个名为 p 的普通函数，并且返回 Screen 类的一个 char 成员。因为它声明的是一个普通函数，所以不能使用 const 限定符。

和普通函数指针不同的是，在成员函数和指向该成员的指针之间不存在自动转换规则：

```
// pmf 指向一个 Screen 成员，该成员不接受任何实参且返回类型是 char
pmf = &Screen::get;           // 必须显式地使用取地址运算符
pmf = Screen::get;           // 错误：在成员函数和指针之间不存在自动转换规则
```

839 使用成员函数指针

和使用指向数据成员的指针一样，我们使用.*或者->*运算符作用于指向成员函数的指针，以调用类的成员函数：

```
Screen myScreen, *pScreen = &myScreen;
// 通过 pScreen 所指的对象调用 pmf 所指的函数
char c1 = (pScreen->*pmf)();
// 通过 myScreen 对象将实参 0, 0 传给含有两个形参的 get 函数
char c2 = (myScreen.*pmf2)(0, 0);
```

之所以(myScreen->*pmf)()和(pScreen.*pmf2)(0,0)的括号必不可少，原因是调用运算符的优先级要高于指针指向成员运算符的优先级。

假设去掉括号的话，

```
myScreen.*pmf()
```

其含义将等同于下面的式子：

```
myScreen.*(pmf())
```

这行代码的意思是调用一个名为 pmf 的函数，然后使用该函数的返回值作为指针指向成员运算符(.*)的运算对象。然而 pmf 并不是一个函数，因此代码将发生错误。



因为函数调用运算符的优先级较高，所以在声明指向成员函数的指针并使用这样的指针进行函数调用时，括号必不可少：(C::*p)(parms) 和 (obj.*p)(args)。

使用成员指针的类型别名

使用类型别名或 typedef（参见 2.5.1 节，第 60 页）可以让成员指针更容易理解。例如，下面的类型别名将 Action 定义为两参数 get 函数的同义词：

```
// Action 是一种可以指向 Screen 成员函数的指针，它接受两个 pos 实参，返回一个 char
using Action =
char (Screen::*)(Screen::pos, Screen::pos) const;
```

Action 是某类型的另外一个名字，该类型是“指向 Screen 类的常量成员函数的指针，其中这个成员函数接受两个 pos 形参，返回一个 char”。通过使用 Action，我们可以简化指向 get 的指针定义：

```
Action get = &Screen::get;           // get 指向 Screen 的 get 成员
```

和其他函数指针类似，我们可以将指向成员函数的指针作为某个函数的返回类型或形参类型。其中，指向成员的指针形参也可以拥有默认实参：

```
// action 接受一个 Screen 的引用，和一个指向 Screen 成员函数的指针
Screen& action(Screen&, Action = &Screen::get);
```

action 是包含两个形参的函数，其中一个形参是 Screen 对象的引用，另一个形参是指向 Screen 成员函数的指针，成员函数必须接受两个 pos 形参并返回一个 char。当我们调用 action 时，只需将 Screen 的一个符合要求的函数的指针或地址传入即可。

840

```
Screen myScreen;
// 等价的调用：
action(myScreen);           // 使用默认实参
action(myScreen, get);      // 使用我们之前定义的变量 get
action(myScreen, &Screen::get); // 显式地传入地址
```



通过使用类型别名，可以令含有成员指针的代码更易读写。

成员指针函数表

对于普通函数指针和指向成员函数的指针来说，一种常见的用法是将其存入一个函数表当中（参见 14.8.3 节，第 511 页）。如果一个类含有几个相同类型的成员，则这样一张表可以帮助我们从中选择一个。假定 Screen 类含有几个成员函数，每个函数负责将光标向指定的方向移动：

```
class Screen {
public:
    // 其他接口和实现成员与之前一致
    Screen& home();           // 光标移动函数
    Screen& forward();
    Screen& back();
    Screen& up();
    Screen& down();
};
```

这几个新函数有一个共同点：它们都不接受任何参数，并且返回值是发生光标移动的 Screen 的引用。

我们希望定义一个 move 函数，使其可以调用上面的任意一个函数并执行对应的操作。为了支持这个新函数，我们将在 Screen 中添加一个静态成员，该成员是指向光标移动函数的指针的数组：

```
class Screen {
public:
    // 其他接口和实现成员与之前一致
    // Action 是一个指针，可以用任意一个光标移动函数对其赋值
    using Action = Screen& (Screen::*)();
    // 指定具体要移动的方向，其中 enum 参见 19.3 节（第 736 页）
```

```
enum Directions { HOME, FORWARD, BACK, UP, DOWN };
Screen& move(Directions);
private:
    static Action Menu[];           // 函数表
};
```

841 数组 Menu 依次保存每个光标移动函数的指针，这些函数将按照 Directions 中枚举成员对应的偏移量存储。move 函数接受一个枚举成员并调用相应的函数：

```
Screen& Screen::move(Directions cm)
{
    // 运行 this 对象中索引值为 cm 的元素
    return (this->*Menu[cm])(); // Menu[cm] 指向一个成员函数
}
```

move 中的函数调用的原理是：首先获取索引值为 cm 的 Menu 元素，该元素是指向 Screen 成员函数的指针。我们根据 this 所指的對象调用该元素所指的成员函数。

当我们调用 move 函数时，给它传入一个表示光标移动方向的枚举成员：

```
Screen myScreen;
myScreen.move(Screen::HOME); // 调用 myScreen.home
myScreen.move(Screen::DOWN); // 调用 myScreen.down
```

剩下的工作就是定义并初始化函数表本身了：

```
Screen::Action Screen::Menu[] = { &Screen::home,
                                   &Screen::forward,
                                   &Screen::back,
                                   &Screen::up,
                                   &Screen::down,
                                   };
```

19.4.2 节练习

练习 19.14：下面的代码合法吗？如果合法，代码的含义是什么？如果不合法，解释原因。

```
auto pmf = &Screen::get_cursor;
pmf = &Screen::get;
```

练习 19.15：普通函数指针和指向成员函数的指针有何区别？

练习 19.16：声明一个类型别名，令其作为指向 Sales_data 的 avg_price 成员的指针的同义词。

练习 19.17：为 Screen 的所有成员函数类型各定义一个类型别名。

842 19.4.3 将成员函数用作可调用对象

如我们所知，要想通过一个指向成员函数的指针进行函数调用，必须首先利用.*运算符或->*运算符将该指针绑定到特定的对象上。因此与普通的函数指针不同，成员指针不是一个可调用对象，这样的指针不支持函数调用运算符（参见 10.3.2 节，第 346 页）。

因为成员指针不是可调用对象，所以我们不能直接将一个指向成员函数的指针传递给算法。举个例子，如果我们想在一个 string 的 vector 中找到第一个空 string，显然

不能使用下面的语句：

```
auto fp = &string::empty; // fp 指向 string 的 empty 函数
// 错误，必须使用.*或->*调用成员指针
find_if(svec.begin(), svec.end(), fp);
```

find_if 算法需要一个可调用对象，但我们提供给它的是一个指向成员函数的指针 fp。因此在 find_if 的内部将执行如下形式的代码，从而导致无法通过编译：

```
// 检查对当前元素的断言是否为真
if (fp(*it)) // 错误：要想通过成员指针调用函数，必须使用->*运算符
```

显然该语句试图调用的是传入的对象，而非函数。

使用 function 生成一个可调用对象

从指向成员函数的指针获取可调用对象的一种方法是使用标准库模板 function（参见 14.8.3 节，第 511 页）：

```
function<bool (const string&)> fcn = &string::empty;
find_if(svec.begin(), svec.end(), fcn);
```

我们告诉 function 一个事实：即 empty 是一个接受 string 参数并返回 bool 值的函数。通常情况下，执行成员函数的对象将被传给隐式的 this 形参。当我们想要使用 function 为成员函数生成一个可调用对象时，必须首先“翻译”该代码，使得隐式的形参变成显式的。

当一个 function 对象包含有一个指向成员函数的指针时，function 类知道它必须使用正确的指向成员的指针运算符来执行函数调用。也就是说，我们可以认为在 find_if 当中含有类似于如下形式的代码：

```
// 假设 it 是 find_if 内部的迭代器，则*it 是给定范围内的一个对象
if (fcn(*it)) // 假设 fcn 是 find_if 内部的一个可调用对象的名字
```

其中，function 将使用正确的指向成员的指针运算符。从本质上来看，function 类将函数调用转换成了如下形式：

```
// 假设 it 是 find_if 内部的迭代器，则*it 是给定范围内的一个对象
if (((*it).*p)()) // 假设 p 是 fcn 内部的一个指向成员函数的指针
```

当我们定义一个 function 对象时，必须指定该对象所能表示的函数类型，即可调用对象的形式。如果可调用对象是一个成员函数，则第一个形参必须表示该成员是在哪个（一般是隐式的）对象上执行的。同时，我们提供给 function 的形式中还必须指明对象是否是以指针或引用的形式传入的。

以定义 fcn 为例，我们想在 string 对象的序列上调用 find_if，因此我们要求 function 生成一个接受 string 对象的可调用对象。又因为我们的 vector 保存的是 string 的指针，所以必须指定 function 接受指针：

```
vector<string*> pvec;
function<bool (const string*)> fp = &string::empty;
// fp 接受一个指向 string 的指针，然后使用->*调用 empty
find_if(pvec.begin(), pvec.end(), fp);
```

使用 mem_fn 生成一个可调用对象

通过上面的介绍可知，要想使用 function，我们必须提供成员的调用形式。我们也

C++
11

可以采取另外一种方法，通过使用标准库功能 `mem_fn` 来让编译器负责推断成员的类型。和 `function` 一样，`mem_fn` 也定义在 `functional` 头文件中，并且可以从成员指针生成一个可调用对象；和 `function` 不同的是，`mem_fn` 可以根据成员指针的类型推断可调用对象的类型，而无须用户显式地指定：

```
find_if(svec.begin(), svec.end(), mem_fn(&string::empty));
```

我们使用 `mem_fn(&string::empty)` 生成一个可调用对象，该对象接受一个 `string` 实参，返回一个 `bool` 值。

`mem_fn` 生成的可调用对象可以通过对象调用，也可以通过指针调用：

```
auto f = mem_fn(&string::empty); // f 接受一个 string 或者一个 string*
f(*svec.begin());               // 正确：传入一个 string 对象，f 使用.*调用 empty
f(&svec[0]);                     // 正确：传入一个 string 的指针，f 使用->*调用 empty
```

实际上，我们可以认为 `mem_fn` 生成的可调用对象含有一对重载的函数调用运算符：一个接受 `string*`，另一个接受 `string&`。

使用 bind 生成一个可调用对象

出于完整性的考虑，我们还可以使用 `bind`（参见 10.3.4 节，第 354 页）从成员函数生成一个可调用对象：

```
// 选择范围内的每个 string，并将其 bind 到 empty 的第一个隐式实参上
auto it = find_if(svec.begin(), svec.end(),
                  bind(&string::empty, _1));
```

和 `function` 类似的地方是，当我们使用 `bind` 时，必须将函数中用于表示执行对象的隐式形参转换成显式的。和 `mem_fn` 类似的地方是，`bind` 生成的可调用对象的第一个实参既可以是 `string` 的指针，也可以是 `string` 的引用：

```
auto f = bind(&string::empty, _1);
f(*svec.begin()); // 正确：实参是一个 string，f 使用.*调用 empty
f(&svec[0]);       // 正确：实参是一个 string 的指针，f 使用->*调用 empty
```

19.4.3 节练习

练习 19.18：编写一个函数，使用 `count_if` 统计在给定的 `vector` 中有多少个空 `string`。

练习 19.19：编写一个函数，令其接受 `vector<Sales_data>` 并查找平均价格高于某个值的第一个元素。

19.5 嵌套类

一个类可以定义在另一个类的内部，前者称为**嵌套类**（`nested class`）或**嵌套类型**（`nested type`）。嵌套类常用于定义作为实现部分的类，比如我们在文本查询示例中使用的 `QueryResult` 类（参见 12.3 节，第 430 页）。

844

嵌套类是一个独立的类，与外层类基本没什么关系。特别是，外层类的对象和嵌套类的对象是相互独立的。在嵌套类的对象中不包含任何外层类定义的成员；类似的，在外层类的对象中也不包含任何嵌套类定义的成员。